

SocialEase – Social Anxiety Helper Web Application

Requirements and Initial Design Document

Team Members: Snaa Elsayy, Rana Mostafa, Miral Adel

Group: SE1

Ai Assisted

1. Brief Project Description

SocialEase is an interactive web application designed to help individuals overcome social anxiety through personalized exercises, anxiety assessments, and progress tracking. The system simulates real-life social scenarios, provides daily confidence-building tasks, and offers tailored recommendations based on user progress. By combining evidence-based techniques with an intuitive interface, SocialEase empowers users to gradually improve their communication skills and reduce social anxiety in a supportive, self-paced environment.

2. Functional Requirements

User Module

- **User Registration:** New users can create an account with personal details
- **User Login:** Registered users can authenticate and access their account
- **Profile Management:** Users can view and update their profile information
- **Logout:** Users can securely log out of the system

Assessment Module

- **Anxiety Assessment:** Users can take standardized anxiety level assessments
- **Score Calculation:** System automatically calculates anxiety score based on responses
- **Assessment History:** Users can view their past assessment results
- **Trend Analysis:** System shows anxiety level trends over time

Exercise Module

- **Browse Exercises:** Users can view available exercises by type and difficulty
- **Start Exercise:** Users can begin an exercise session
- **Complete Exercise:** Users can mark exercises as completed with scores
- **Exercise Categories:** System provides exercises for breathing, conversation, public speaking, eye contact, and role-play

Progress Tracker Module

- **Progress Summary:** Users can view overall progress statistics
- **Progress History:** Users can see detailed progress records over time
- **Metric Recording:** System records various confidence and anxiety metrics
- **Streak Tracking:** System tracks consecutive days of exercise completion
- **Visualization:** Progress data is presented in an easy-to-understand format

Recommendation System Module

- **Personalized Recommendations:** System suggests exercises based on user progress and anxiety levels
 - **Accept Recommendations:** Users can accept suggested exercises
 - **Dismiss Recommendations:** Users can dismiss irrelevant suggestions
 - **Reason Explanation:** System provides reasoning for each recommendation
-

3. User Stories

1. **As a new user**, I want to register for an account so that I can access the SocialEase platform and start my journey to overcome social anxiety.
 2. **As a registered user**, I want to take an anxiety assessment so that I can understand my current anxiety level and track improvements over time.
 3. **As a user**, I want to browse available exercises by category (breathing, conversation, public speaking) so that I can choose exercises that match my current needs and comfort level.
 4. **As a user**, I want to complete exercises and receive scores so that I can track my performance and see my improvement.
 5. **As a user**, I want to view my progress summary and history so that I can see how much I've improved and stay motivated to continue.
 6. **As a user**, I want to receive personalized exercise recommendations so that I can focus on areas where I need the most improvement.
 7. **As a user**, I want to update my profile information so that my account details remain accurate and up-to-date.
-

4. Non-Functional Requirements

Performance

- The system should respond to user requests within 2 seconds under normal load
- The application should support up to 100 concurrent users without performance degradation
- Database queries should be optimized to return results within 500ms

Security

- User passwords must be encrypted using strong hashing algorithms (BCrypt)
- All API endpoints except login/register must require authentication via JWT tokens
- User data must be protected and only accessible by the authenticated user
- Input validation must be implemented to prevent SQL injection and XSS attacks

Usability

- The interface should be intuitive and easy to navigate for users with varying technical skills
- The application should be responsive and work well on both desktop and mobile devices
- Clear error messages should be displayed to guide users when issues occur
- Exercise instructions should be clear and easy to understand

Reliability

- The system should have 99% uptime during active usage hours
- Data should be backed up regularly to prevent loss
- The application should handle errors gracefully without crashing

Scalability

- The architecture should allow for easy addition of new exercise types and assessment questions
- The database schema should be designed to handle growing amounts of user data
- The system should be deployable on cloud infrastructure for horizontal scaling

5. System Constraints and Assumptions

Constraints

- The application will be developed using Java Spring Boot for the backend
- PostgreSQL will be used as the database management system
- The frontend will be built using HTML, CSS, and JavaScript (React optional)
- The system will be deployed as a web application, not a mobile native app
- Internet connectivity is required to access the application

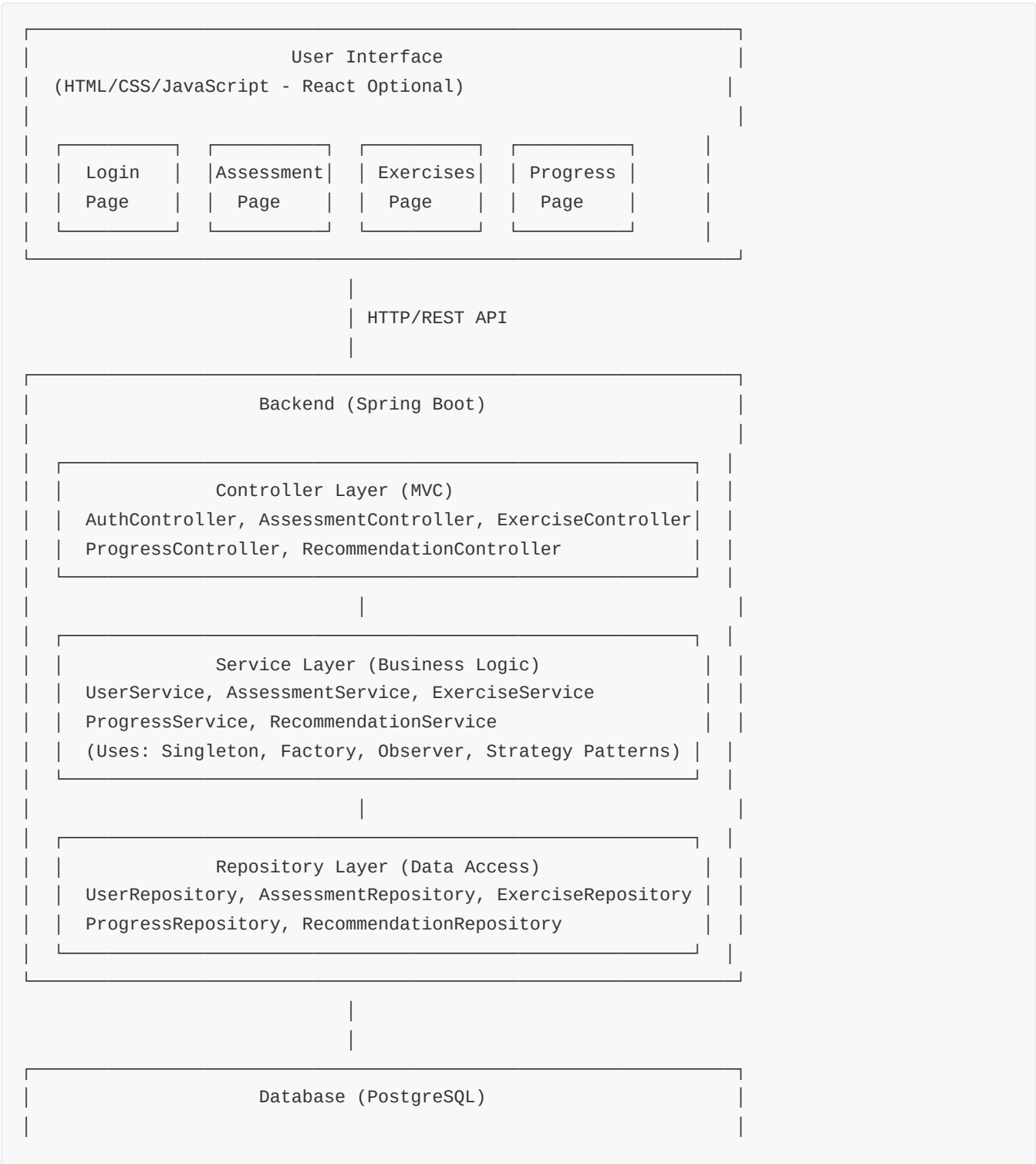
Assumptions

- Users have basic computer literacy and can navigate web applications
- Users will provide honest responses during anxiety assessments
- The system will be used primarily by individuals with mild to moderate social anxiety
- Instructors and mental health professionals are not directly involved in the system (self-guided)
- Users have access to a device with a modern web browser

Dependencies

- Reliable internet connection for users
- Spring Boot framework and related libraries
- PostgreSQL database management system
- JWT library for authentication
- Frontend framework/libraries (if React is chosen)

6. Initial High-Level Architecture Diagram





7. Selected Design Patterns

Pattern 1: Singleton Pattern

Where it will be used:

- Database connection management
- Configuration settings management
- Logger instance management

Why it is suitable:

The Singleton pattern ensures that only one instance of a class exists throughout the application lifecycle. For database connections and configuration management, having multiple instances could lead to resource wastage, connection conflicts, or inconsistent state.

Expected benefit:

- Controlled access to a single instance
- Reduced memory footprint by avoiding multiple instances
- Consistent state management across the application
- Centralized control of shared resources

Implementation example:

```
public class DatabaseConnection {
    private static DatabaseConnection instance;
    private Connection connection;

    private DatabaseConnection() {
        // Initialize connection
    }

    public static synchronized DatabaseConnection getInstance() {
        if (instance == null) {
            instance = new DatabaseConnection();
        }
        return instance;
    }
}
```

Pattern 2: Factory Pattern

Where it will be used:

- Creating different types of exercises (Breathing, Conversation, Public Speaking, Eye Contact, Role-Play)
- Creating different assessment question sets based on anxiety type

Why it is suitable:

The Factory pattern provides an interface for creating objects without specifying their exact classes. Since SocialEase needs to create various types of exercises with different properties and behaviors, the Factory pattern allows for flexible and scalable object creation.

Expected benefit:

- Loose coupling between client code and object creation
- Easy addition of new exercise types without modifying existing code
- Centralized object creation logic
- Improved code maintainability and extensibility

Implementation example:

```
public interface ExerciseFactory {
    Exercise createExercise();
}

public class BreathingExerciseFactory implements ExerciseFactory {
    @Override
    public Exercise createExercise() {
        return new BreathingExercise();
    }
}

public class ExerciseCreator {
    public static Exercise createExercise(String type) {
        switch (type) {
            case "BREATHING":
                return new BreathingExerciseFactory().createExercise();
            case "CONVERSATION":
                return new ConversationExerciseFactory().createExercise();
            // ... other types
        }
    }
}
```

Pattern 3: Observer Pattern

Where it will be used:

- Notifying users about progress updates
- Sending notifications when recommendations are generated

- Alerting users when they achieve milestones or streaks

Why it is suitable:

The Observer pattern defines a one-to-many dependency between objects. When one object changes state, all its dependents are notified. In SocialEase, when a user completes an exercise or reaches a milestone, multiple components (progress tracker, recommendation system, notification system) need to be updated.

Expected benefit:

- Loose coupling between subject and observers
- Dynamic relationship between objects
- Automatic notification of state changes
- Easy addition of new observers without modifying the subject

Implementation example:

```
public interface ProgressObserver {
    void onProgressUpdate(User user, ProgressRecord record);
}

public class ProgressTracker {
    private List<ProgressObserver> observers = new ArrayList<>();

    public void addObserver(ProgressObserver observer) {
        observers.add(observer);
    }

    public void notifyObservers(User user, ProgressRecord record) {
        for (ProgressObserver observer : observers) {
            observer.onProgressUpdate(user, record);
        }
    }
}
```

Pattern 4: Strategy Pattern

Where it will be used:

- Applying different recommendation strategies based on user needs
- Different assessment scoring algorithms based on anxiety type
- Different exercise difficulty adjustment strategies

Why it is suitable:

The Strategy pattern defines a family of algorithms and makes them interchangeable. SocialEase needs to apply different approaches for different users based on their anxiety levels, progress, and preferences. The Strategy pattern allows the system to switch between different algorithms dynamically.

Expected benefit:

- Elimination of conditional statements for algorithm selection
- Easy swapping of algorithms at runtime

- Improved code organization and readability
- Adherence to the Open/Closed Principle

Implementation example:

```
public interface RecommendationStrategy {
    List<Exercise> recommendExercises(User user, List<Assessment> assessments);
}

public class AnxietyBasedStrategy implements RecommendationStrategy {
    @Override
    public List<Exercise> recommendExercises(User user, List<Assessment> assessments) {
        // Recommend based on anxiety level
    }
}

public class ProgressBasedStrategy implements RecommendationStrategy {
    @Override
    public List<Exercise> recommendExercises(User user, List<Assessment> assessments) {
        // Recommend based on past progress
    }
}

public class RecommendationService {
    private RecommendationStrategy strategy;

    public void setStrategy(RecommendationStrategy strategy) {
        this.strategy = strategy;
    }

    public List<Exercise> getRecommendations(User user) {
        return strategy.recommendExercises(user, getAssessments(user));
    }
}
```

Pattern 5: MVC (Model-View-Controller) Pattern

Where it will be used:

- Organizing the entire application structure
- Separating user interface (View) from business logic (Controller) and data (Model)

Why it is suitable:

MVC is a fundamental architectural pattern that separates an application into three main logical components. Spring Boot naturally supports MVC, making it ideal for organizing the SocialEase application. This separation allows for parallel development, easier testing, and better maintainability.

Expected benefit:

- Clear separation of concerns
- Improved code organization

- Easier testing and debugging
- Support for parallel development by team members
- Natural fit with Spring Boot framework

8. Initial Class List

Model Classes (Entities)

Class Name	Responsibility
User	Represents a registered user with personal information, credentials, and profile data
Assessment	Represents an anxiety assessment with responses, score, and anxiety level
Exercise	Represents an exercise with title, description, type, difficulty, and duration
UserExercise	Represents a user's interaction with an exercise (status, score, completion date)
ProgressRecord	Represents a progress metric recorded for a user at a specific time
Recommendation	Represents a personalized exercise recommendation for a user

Controller Classes

Class Name	Responsibility
AuthController	Handles user registration, login, and authentication requests
UserController	Manages user profile operations (view, update)
AssessmentController	Handles assessment creation, retrieval, and history requests
ExerciseController	Manages exercise browsing, starting, and completion
ProgressController	Handles progress summary, history, and metric recording requests
RecommendationController	Manages recommendation retrieval, acceptance, and dismissal

Service Classes

Class Name	Responsibility
UserService	Contains business logic for user management and authentication
AssessmentService	Handles assessment logic, scoring, and trend analysis
ExerciseService	Manages exercise retrieval, filtering, and completion logic
ProgressService	Handles progress tracking, metric calculation, and history retrieval

Class Name	Responsibility
RecommendationService	Generates personalized recommendations using strategy pattern

Repository Classes (Data Access)

Class Name	Responsibility
UserRepository	Provides data access methods for User entity
AssessmentRepository	Provides data access methods for Assessment entity
ExerciseRepository	Provides data access methods for Exercise entity
UserExerciseRepository	Provides data access methods for UserExercise entity
ProgressRecordRepository	Provides data access methods for ProgressRecord entity
RecommendationRepository	Provides data access methods for Recommendation entity

Factory Classes

Class Name	Responsibility
ExerciseFactory	Interface for creating different types of exercises
BreathingExerciseFactory	Creates breathing exercise instances
ConversationExerciseFactory	Creates conversation exercise instances
PublicSpeakingExerciseFactory	Creates public speaking exercise instances
ExerciseCreator	Central class that uses factories to create exercises

Strategy Classes

Class Name	Responsibility
RecommendationStrategy	Interface for different recommendation algorithms
AnxietyBasedStrategy	Recommends exercises based on user's anxiety level
ProgressBasedStrategy	Recommends exercises based on user's past progress
CombinedStrategy	Combines multiple factors for recommendations

Observer Classes

Class Name	Responsibility
ProgressObserver	Interface for objects that observe progress changes
NotificationService	Sends notifications when progress is updated
RecommendationUpdater	Updates recommendations when user completes exercises
ProgressTracker	Subject that notifies observers of progress changes

Utility/Configuration Classes

Class Name	Responsibility
DatabaseConnection	Singleton class managing database connections
JwtTokenUtil	Utility class for JWT token generation and validation
SecurityConfig	Configuration class for Spring Security setup
ApplicationConfig	Main configuration class for the application